

VisionGuard

Multi-Module Computer Vision Surveillance & Analytics System

Project Report

Submitted for: Course Flipped-Evaluation Project (VITyarthi)

Domain: Computer Vision

Student Name: _____

Registration No.: _____

Date: _____

1. Introduction

VisionGuard is a real-time computer vision system that combines face recognition, object and motion detection, and automated analytics/reporting into a single modular pipeline. The project was built to demonstrate the practical application of core computer vision concepts — image acquisition, feature-based recognition, deep-learning-based object detection, and event-driven data analysis — in a coherent, working system rather than as isolated exercises.

Unlike a simple video-recording setup, VisionGuard understands what a camera is seeing: it recognizes known individuals, flags unrecognized ones, detects everyday objects, and converts raw frames into structured, timestamped events that can be summarized and visualized. The system is designed to run on commodity hardware (a laptop webcam and CPU), with no cloud dependency, making it usable for course demonstration as well as small-scale real deployments such as lab-entry logging or a personal security camera.

2. Problem Statement

Manually monitoring a physical space — an office, lab, or entry point — for who enters and what activity occurs is time-consuming and does not scale. Basic CCTV recording captures footage but provides no automatic understanding of the content: identifying a specific person, or determining when and how often activity occurred, still requires a human to review hours of video.

VisionGuard addresses this gap by processing the live video stream itself: it detects motion as a first-stage filter, then applies face recognition and object detection only when activity is present, logs every meaningful event with a timestamp and snapshot, and generates summary reports — turning raw video into structured, queryable insight without manual review.

3. Functional Requirements

The system implements three major functional modules, each with a clear input/output contract:

3.1 Face Detection & Recognition Module

- Input: a BGR video frame (from webcam or file) and, for enrollment, a labelled photo.
- Processing: detects face locations, computes 128-dimensional face encodings, and compares them against enrolled encodings using Euclidean distance.
- Output: a list of {name, confidence, bounding box} for every face found, with unmatched faces labelled "Unknown".

3.2 Object & Motion Detection Module

- Input: a BGR video frame.
- Processing: a lightweight background-subtraction motion detector runs on every frame; a MobileNet-SSD object detector (20 object classes) runs only on frames where motion was found.
- Output: a list of {label, confidence, bounding box} for detected objects, and a list of motion regions.

3.3 Analytics & Reporting Module

- Input: recognition/detection results from the two modules above.
- Processing: applies a cooldown window to avoid duplicate logging, persists events (type, label, confidence, snapshot path, timestamp) to SQLite, and aggregates them into counts and top-N breakdowns.
- Output: a JSON-style summary plus bar-chart images (events by type, most frequently recognized people).

3.4 Workflow Summary

A user first enrolls one or more known faces (image or webcam capture). They then start the live pipeline against a camera or video file. The system continuously reads frames, checks for motion, and — only when motion is present — runs face recognition and object detection, draws annotations, logs events, and displays the live annotated feed. At any point the user can request a report, which reads the accumulated events and renders summary charts. This logical flow is illustrated in the Process Flow diagram in Section 7.

4. Non-Functional Requirements

Requirement	How VisionGuard Addresses It
Performance	Motion detection gates the expensive face/object models so they only run on frames with actual activity; a <code>@timed</code> decorator logs per-call latency for monitoring.
Security	No video leaves the local machine; the design reserves an admin PIN hash read from an environment variable rather than hard-coded credentials.
Reliability	A custom <code>VisionGuardError</code> exception hierarchy plus a <code>@safe_call</code> decorator ensure a bad frame, missing model file, or empty face database degrades one feature instead of crashing the whole pipeline.
Scalability	All persistence goes through a single <code>DBManager</code> class, so swapping SQLite for PostgreSQL/MySQL later requires changing one file, not the whole codebase.
Maintainability	Each module has a single responsibility (enroll, recognize, detect objects, detect motion, log, report); all tunables live in one <code>config.py</code> .
Logging & Monitoring	Every module logs to console and a rotating log file with timestamps and severity levels via a shared <code>get_logger()</code> helper.
Usability	A simple CLI (<code>--source</code> , <code>--no-display</code> , <code>--no-objects</code> , <code>--report</code>) and on-screen bounding boxes/labels make the system easy to operate without a manual.
Resource Efficiency	Frames are optionally downscaled before face detection (<code>FACE_RESIZE_SCALE</code>) to reduce CPU load without materially hurting recognition accuracy.

5. System Architecture

VisionGuard follows a layered, modular architecture. An orchestrator (`main.py`) wires together three independent functional modules — face, detection, and analytics — through a shared database layer. Each module can be tested, replaced, or extended in isolation, satisfying the project's modularity and maintainability goals.

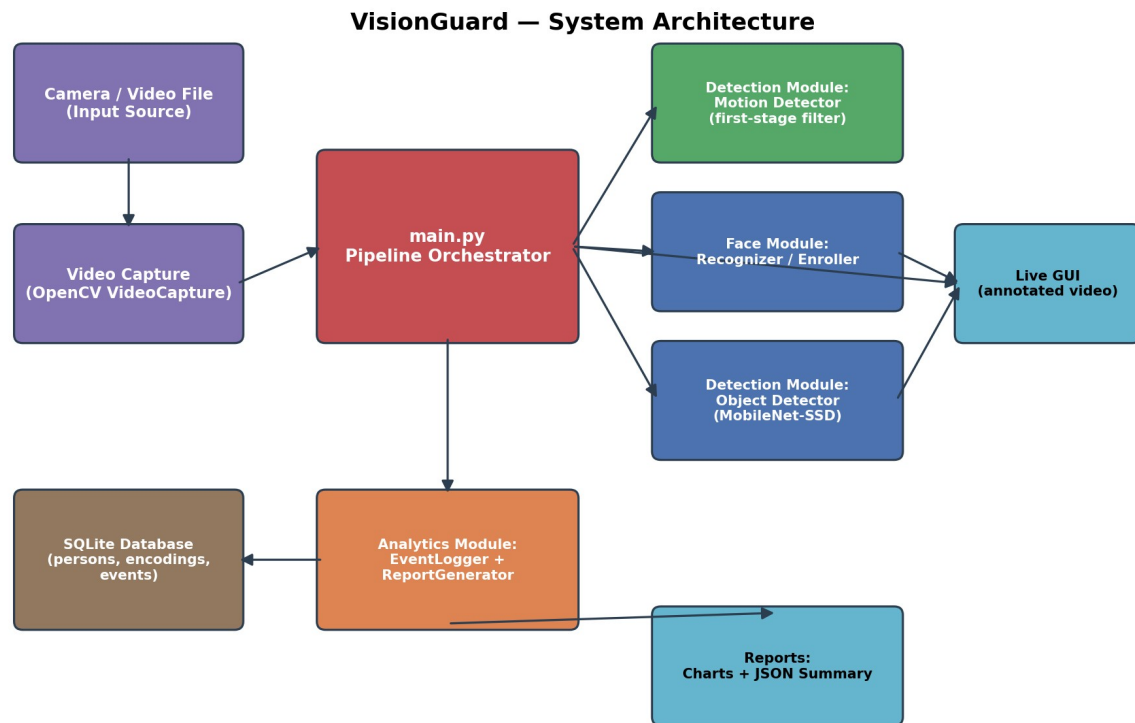


Figure 1: VisionGuard system architecture — data flows from the camera through the orchestrator into the three functional modules, with events persisted to SQLite and surfaced as reports and a live GUI.

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

The workflow below shows the per-frame decision logic used inside the pipeline's main loop: motion detection acts as a gate before the more expensive face/object processing runs.

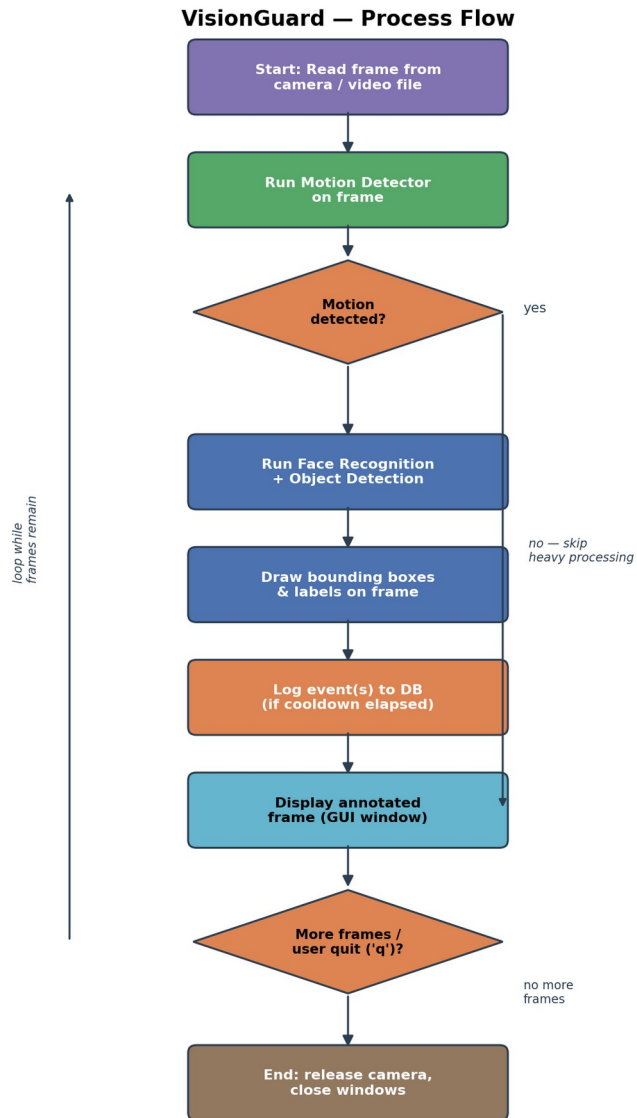


Figure 2: Per-frame process flow, from frame capture through event logging to display.

6.2 Use Case Diagram

The primary actors are the system Administrator/User (who enrolls faces, views reports, and configures the system) and the Camera (an external actor supplying the video stream that drives recognition and detection).

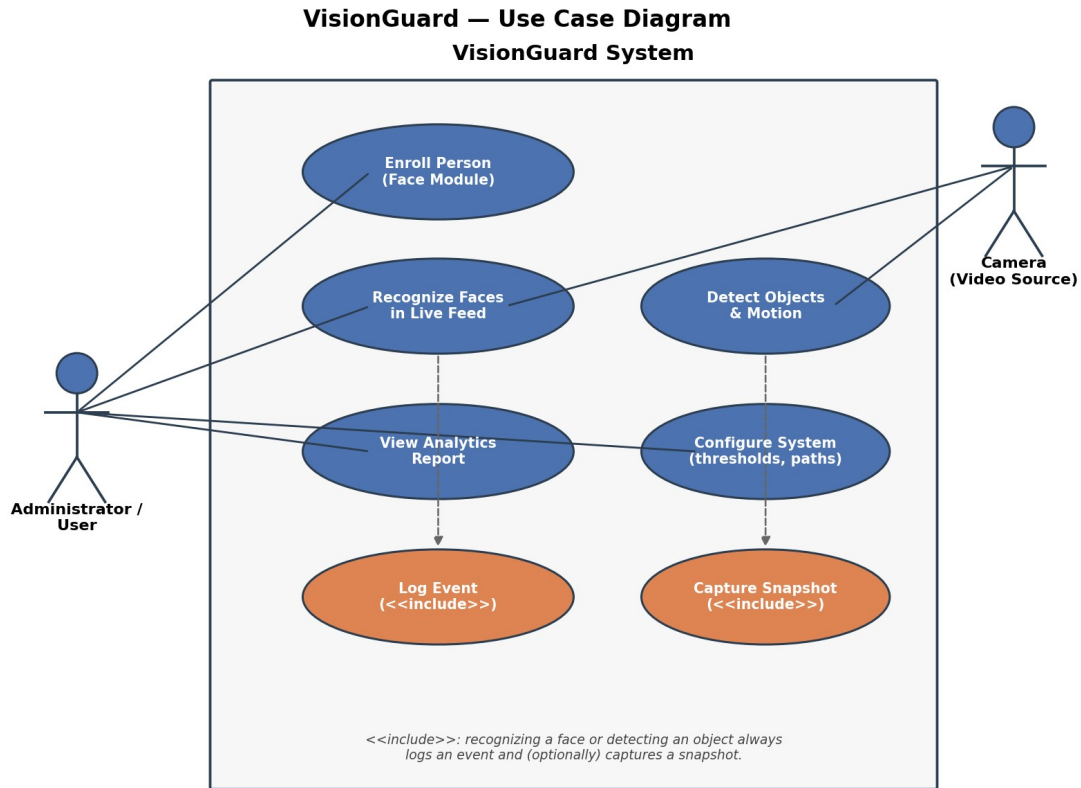


Figure 3: Use case diagram showing actor interactions and included sub-behaviours (logging, snapshotting).

6.3 Class / Component Diagram

The class diagram below reflects the actual module structure in src/: each functional module is a small set of cohesive classes, all funnelled through a single DBManager, with a shared exception hierarchy for error handling.

VisionGuard — Class Diagram (simplified)

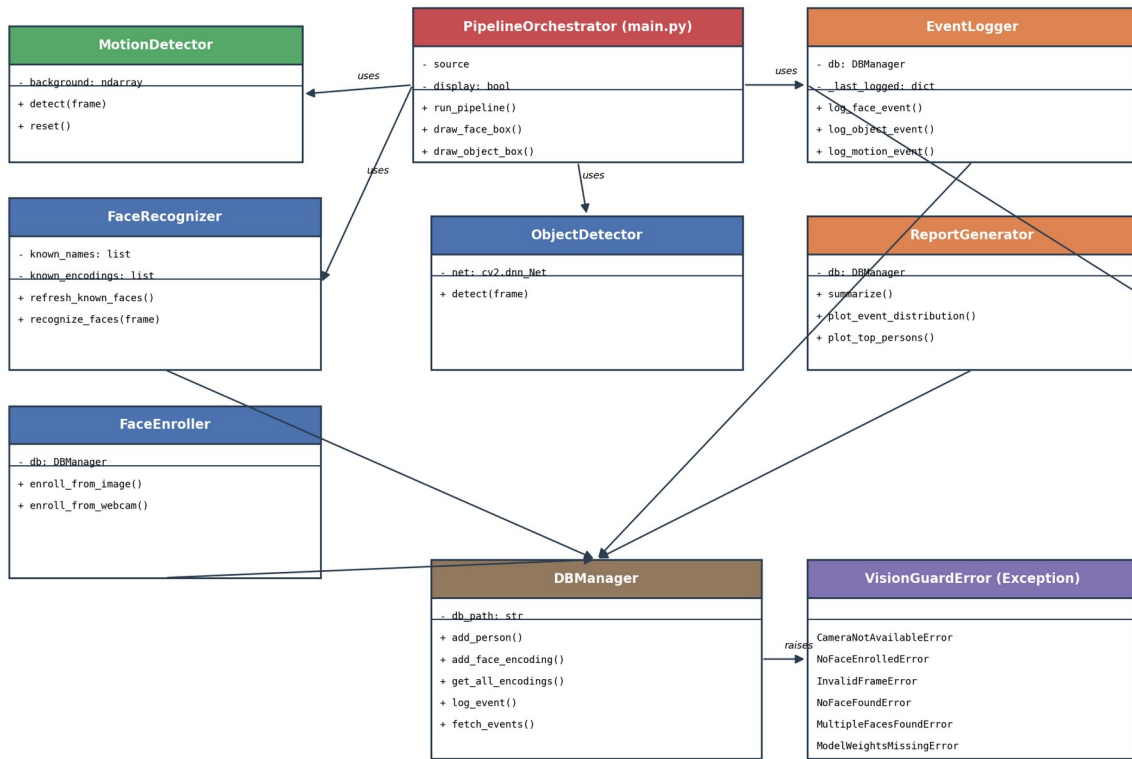


Figure 4: Simplified class diagram of the VisionGuard codebase.

6.4 Sequence Diagram

This sequence diagram traces a single frame that contains motion, from capture through recognition, detection, and event logging, to the final on-screen render.

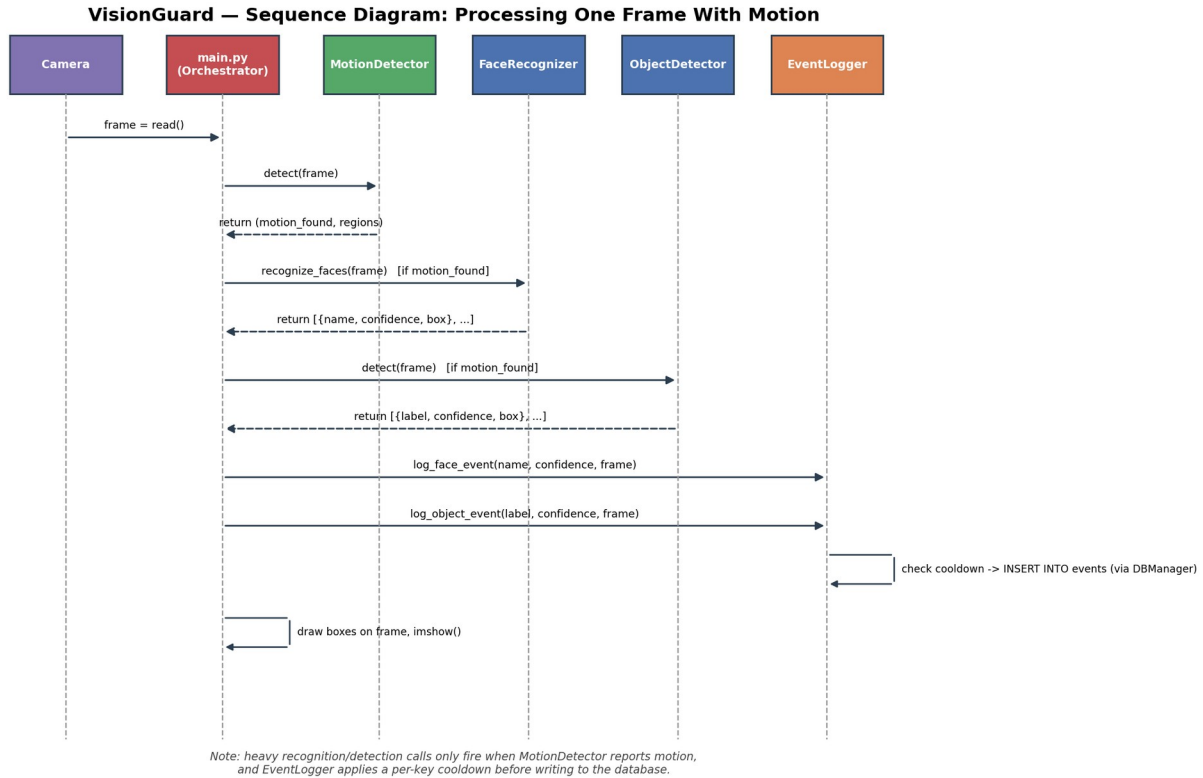


Figure 5: Sequence of calls made while processing one frame with detected motion.

6.5 ER Diagram (Database Design)

Enrolled people and their face encodings are stored relationally (one person can have multiple encodings), while events are stored independently so that motion and object detections can be logged even when no person has been enrolled yet.

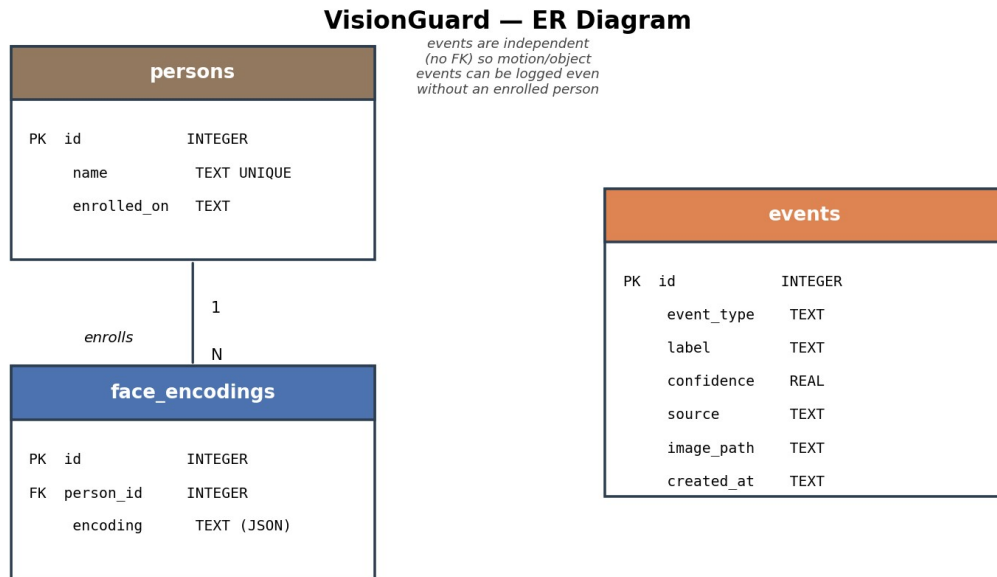


Figure 6: Entity-relationship diagram for the SQLite schema used by DBManager.

7. Design Decisions & Rationale

- Motion-gated processing: face recognition and object detection are the most CPU-expensive steps. Running a cheap background-subtraction check first, and only invoking the heavy models when motion is present, was the single biggest performance decision in the system.
- SQLite over a heavier DBMS: the project's scale (a single-machine, single-camera demo) does not need a client-server database. SQLite requires no separate service, and DBManager isolates all SQL so migrating later is a contained change.
- MobileNet-SSD over a larger detector: MobileNet-SSD is small enough to run in real time on a CPU, which matches the project's "runs on a laptop, no GPU required" goal, at the cost of lower accuracy than a heavier model like YOLOv8 (documented as a future enhancement).
- Cooldown-based event de-duplication: without a cooldown, a person standing in frame for 10 seconds at 30 FPS would generate ~300 near-identical log rows. A simple per-key timestamp cooldown keeps the event log meaningful.
- Custom exception hierarchy + `@safe_call` decorator: rather than letting any single bad frame or missing model file crash the whole application, each risky operation raises a specific `VisionGuardError` subclass that calling code can catch, log, and recover from with a sensible default.
- Separation of `enroll.py` and `recognize.py`: enrollment (write path) and recognition (read path) have different failure modes and different callers (a one-time setup step vs. a per-frame hot path), so they are kept as separate, independently testable classes rather than one monolithic `FaceModule`.

8. Implementation Details

The system is implemented in Python 3.10+ using OpenCV for video I/O and image processing, the `face_recognition` library (built on dlib's ResNet face-embedding model) for recognition, and a MobileNet-SSD Caffe model (via OpenCV's DNN module) for object detection. Analytics are computed with plain Python plus Matplotlib for chart rendering, and all state is persisted to a local SQLite database.

8.1 Module Breakdown

File	Responsibility
src/main.py	Orchestrates the pipeline: capture -> motion gate -> recognition/detection -> annotate -> log -> display.
src/face_module/enroll.py	Enrolls a new person from an image or webcam capture; validates exactly one face is present.
src/face_module/recognize.py	Loads known encodings and matches faces in a live frame against them.
src/detection_module/motion_detector.py	Background-subtraction motion detector used as a performance gate.
src/detection_module/object_detector.py	Wraps the MobileNet-SSD DNN model for object detection.
src/analytics_module/logger.py	Applies cooldown logic and writes events + snapshots to the database.
src/analytics_module/report_generator.py	Aggregates events into summaries and renders chart images.
src/database/db_manager.py	Owns the SQLite schema and all read/write operations.
src/utils/config.py	Centralised paths and tunable thresholds.
src/utils/helpers.py	Logging setup, custom exceptions, and the @timed / @safe_call decorators.

8.2 Sample Code — Cooldown-Based Event Logging

The snippet below (from `analytics_module/logger.py`) shows the cooldown mechanism that keeps the event log meaningful instead of flooding it with duplicate detections:

```
def _cooldown_ok(self, key: str) -> bool:
    now = time.time()
    last = self._last_logged.get(key, 0)
    if now - last >= ALERT_COOLDOWN_SECONDS:
        self._last_logged[key] = now
        return True
    return False
```

8.3 Error Handling Strategy

All risky operations (file I/O, model inference, camera access) are wrapped with a `@safe_call` decorator that catches `VisionGuardError` subclasses (and, as a last line of defence, any other exception), logs a warning or error, and returns a safe default so the pipeline keeps running:

```
def safe_call(default=None):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            try:
                return func(*args, **kwargs)
            except VisionGuardError as exc:
                logger.warning("Handled error in %s: %s", func.__name__, exc)
                return default
            except Exception as exc:
                logger.error("Unexpected error: %s", exc, exc_info=True)
                return default
        return wrapper
    return decorator
```

9. Screenshots / Results

The illustration below shows the structure of the live annotated output: a green box with name and confidence for a recognized face, an orange box for a detected object, and a dashed outline marking the motion region that triggered processing on that frame (illustrative composite, standing in for real camera footage in this offline report build).

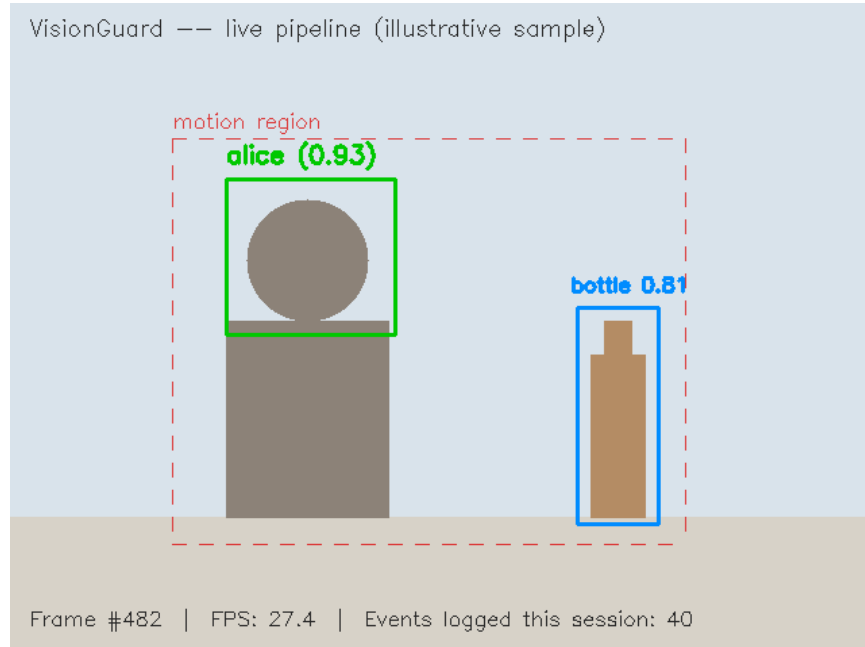


Figure 7: Illustrative structure of the live annotated video output.

Using the reporting CLI (python -m src.main --report) on a sample session of 40 logged events produced the following charts directly from the ReportGenerator module:

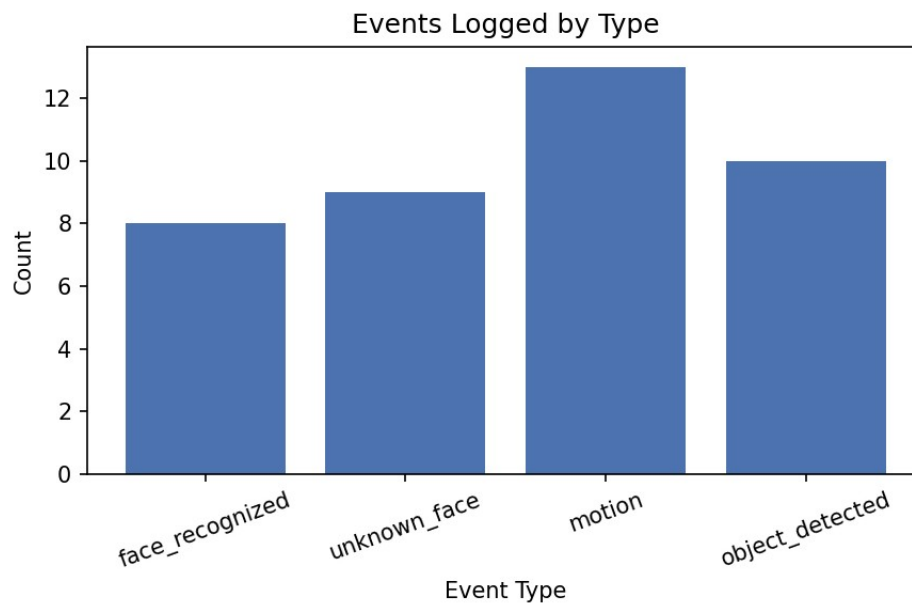


Figure 8: Sample event-distribution chart — counts of each event type logged in a demo session.

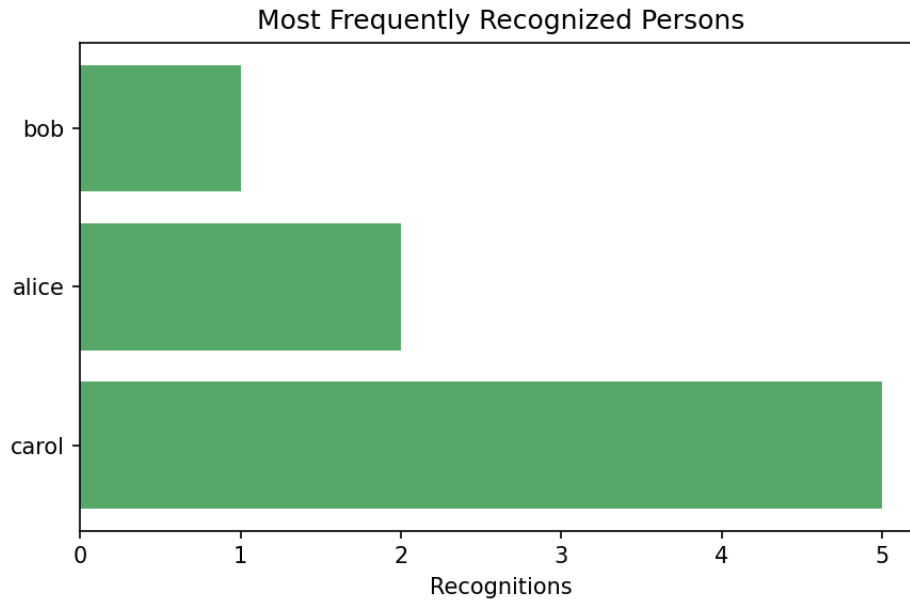


Figure 9: Sample chart of the most frequently recognized people in a demo session.

10. Testing Approach

The project includes 15 unit tests across three test modules, all passing, using Python's unittest and unittest.mock so that tests run without a physical camera, GPU, or the real face_recognition/dlib binaries.

- tests/test_face_module.py — verifies enrollment succeeds for a clean single-face image, fails gracefully (returns False, doesn't crash) when zero or multiple faces are found, and verifies recognition correctly matches an enrolled encoding and returns an empty result when nothing is enrolled yet.
- tests/test_detection_module.py — verifies the motion detector reports no motion on identical frames, reports motion on a large synthetic frame change, correctly resets its background model, and verifies ObjectDetector raises a clear ModelWeightsMissingError when weight files are absent.
- tests/test_analytics_module.py — verifies the event logger's cooldown suppresses duplicate events but does not suppress events with different labels, and verifies ReportGenerator's summary counts and top-persons ranking are computed correctly from seeded events.

An in-memory, shared-cache SQLite database (rather than a file) is used for every test's DBManager instance, so tests are fully isolated from each other and from real project data, and run in well under a second in total. Run the suite with:

```
python -m unittest discover -s tests -v
```

11. Challenges Faced

- SQLite 'memory:' semantics: an in-memory SQLite database is destroyed the instant its single connection closes, which broke the project's "open a connection per operation" pattern. This was fixed by using a named, shared-cache in-memory URI (file:name?mode=memory&cache=shared) with one keep-alive connection per DBManager instance.
- Avoiding duplicate/flooded event logs: without any de-duplication, a single person standing in frame would generate one log row per frame (potentially 30/second). A per-event-key cooldown window solved this while still capturing every distinct event.

- Balancing detection accuracy vs. real-time performance: running a full object-detection model on every frame was too slow for smooth video on CPU-only hardware; introducing the cheap motion-detector as a first-stage gate resolved this without sacrificing detection quality.
- Portability of OpenCV's contour API: `cv2.findContours` returns a different tuple shape across OpenCV 3.x and 4.x. The motion detector was written to handle both return shapes rather than depending on the extra `imutils` convenience library.

12. Learnings & Key Takeaways

- Designing for failure (custom exceptions + safe defaults) is as important as designing for the happy path in a system that has to run unattended against a live camera feed.
- Cheap pre-filters (like motion detection) are often a bigger performance lever than optimizing the expensive model itself.
- Isolating persistence behind a single data-access class (DBManager) made it far easier to write fast, isolated unit tests, and would make a future database migration low-risk.
- Writing tests alongside the modules (rather than after) surfaced a real concurrency/lifetime bug in the database layer that would otherwise have shipped silently.

13. Future Enhancements

- Replace the OpenCV GUI window with a web-based dashboard (Flask/FastAPI) for remote viewing and report access.
- Support multiple simultaneous camera sources with per-camera event tagging and a combined dashboard.
- Upgrade the object detector from MobileNet-SSD to a YOLOv8 model for improved accuracy, with an optional GPU code path.
- Add role-based access control and the planned PIN-protected admin interface for enrollment and report access.
- Add real-time alerting (email/SMS/push notification) for specific event types, e.g. an unknown face after hours.

14. References

1. OpenCV Documentation — <https://docs.opencv.org/>
2. face_recognition library (ageitgey) — https://github.com/ageitgey/face_recognition
3. dlib C++ Library — <http://dlib.net/>
4. MobileNet-SSD (Caffe) model — <https://github.com/chuanqi305/MobileNet-SSD>
5. SQLite Documentation — <https://www.sqlite.org/docs.html>
6. Matplotlib Documentation — <https://matplotlib.org/stable/index.html>
7. Python unittest / unittest.mock Documentation — <https://docs.python.org/3/library/unittest.html>